Symphony and Beethoven Taxonomy Spec

Oct 15, 2026 · @f

1. Purpose and scope

This spec says what kind of software Symphony and Beethoven are, what they sit beside, what they add, and what is still unproven. It is the reference the manual's deep research starts from, and the research corrects it.

It places three things:

- **Symphony**, the coordination layer: a claim protocol, a board and workers for Claude Code agents, kept in a GitHub repository and run on GitHub Actions.
- **Beethoven**, the research layer: protocols for grid jobs on Wayne State's cluster, reproducible CLAIM and REFORMS workflows, and checks on code that agents wrote.
- **The stack** they make together, in which Claude iterates the scientific method inside strict frameworks while Symphony coordinates the agents.

Out of scope: the price of the paid SymphonyBoard App, legal opinions on terms of service (raised as open questions only), and the AIDaS grant text, which takes nothing from the coordination experiment until you say so.

Every claim carries one grade:

Grade	Means	What it needs
Verified	Checked against a primary source opened on the as-of date	A cited link or a file and line
Inferred	Follows from verified facts	The reasoning, written beside it
Reported	Taken from a secondary source or a search summary, because the primary was unreachable from the research session	A re-read at the primary before anyone quotes it
Open	Not yet checked, or unknown	A research question in section 11 that owns it

2. The systems

Symphony: one repository as the shared plane for many agents

Symphony makes one GitHub repository the shared state for any number of Claude Code workers: a person's local or cloud sessions, routines, and agents running as GitHub Actions jobs. It has no server of its own. The rules are a Python library with no dependencies (symphony-protocol, public, MIT), vendored into each repository beside eleven workflow files and Claude Code hooks in a whole-template copy, or beside at most three workflow files (board, tidy and metrics) and the symphony plugin's hooks in a protocol-only adoption. Verified: PROTOCOL.md, research note 01.

Part	What it is	Runs on
Queue	Open issues labelled todo	GitHub Issues
Claim	The push that creates branch claude/N (of two racing creations, git lets exactly one win), whose first commit adds a claim file naming the worker, the plan and the files to touch	git refs on GitHub
Board	STATE.md on the branch board, rewritten on every issue, pull request and branch event, and every six hours as a fallback	board.yml, on the workflow's own token
Release	A claim with no commit for 24 hours loses its labels, gets a comment saying what the branch holds, and its branch is renamed released/N-sha so the work is kept	The board's run
Overlaps	Each file that two pieces of open work change, with the order they should land in (a blocker, then ready work, then the oldest)	The board's run
Workers	Each named worker:name: a person's sessions by GitHub login; routines and Actions workers registered by a file under .github/symphony/workers/	Sessions, routines, workers.yml

Part	What it is	Runs on
Coordinator	Deterministic steps release idle claims and announce overlaps; Claude plans epics into tasks, settles blocked tasks and integrates finished epics	board.yml; coordinator.yml
Gate	The required check named check, which includes the protocol check; a second required check, qa, is planned	Actions and a ruleset
Push guard	A PreToolUse hook that refuses forced pushes, pushes to the default branch and leases onto another worker's claim	Each Claude Code session

Claude runs in only three workflows (claude.yml, the work job of workers.yml and four jobs of coordinator.yml), all through anthropics/claude-code-action@v1 on one subscription token, CLAUDE_CODE_OAUTH_TOKEN. Everything else is deterministic code on the workflow's GITHUB_TOKEN, so a worker with nothing to do starts no Claude session. The design rests on about 30 GitHub primitives, among them concurrency queues, branch renames that close a pull request, and environments with required reviewers. Verified: research note 01.

Maturity. The library's first commit is 2026-10-10 and release 0.4.1 is of 2026-10-13. Its 1,379 tests and the template's 581 passed on 2026-10-15. A validation harness drives the shipped library through every interleaving of scripted workers against an oracle written from the spec's text: 16.7 million states at depth 8, and 18.8 million with two issues at depth 7, with no violation in the run of 2026-10-14, which searched the library before the fixes of F2, F15, F17, F18 and F19; 93 mutants prove the checks bite. Building the harness and reviewing the fixes it led to found 19 defects: six are fixed in release 0.4.1, eleven more on main ahead of the next release, one (F16) was settled by changing the spec, and one (F8, idleness read from the committer date) waits on the owner (validation/DESIGN.md and known.json). Not built yet: the qa check, the cap of 10 Claude runs a day, the public runtime, the control panel, the account pool, and a benchmark run of Symphony at full scale. Verified: validation run of record, research notes 01 and 12.

Beethoven: protocols, gates and agents for research on a university cluster

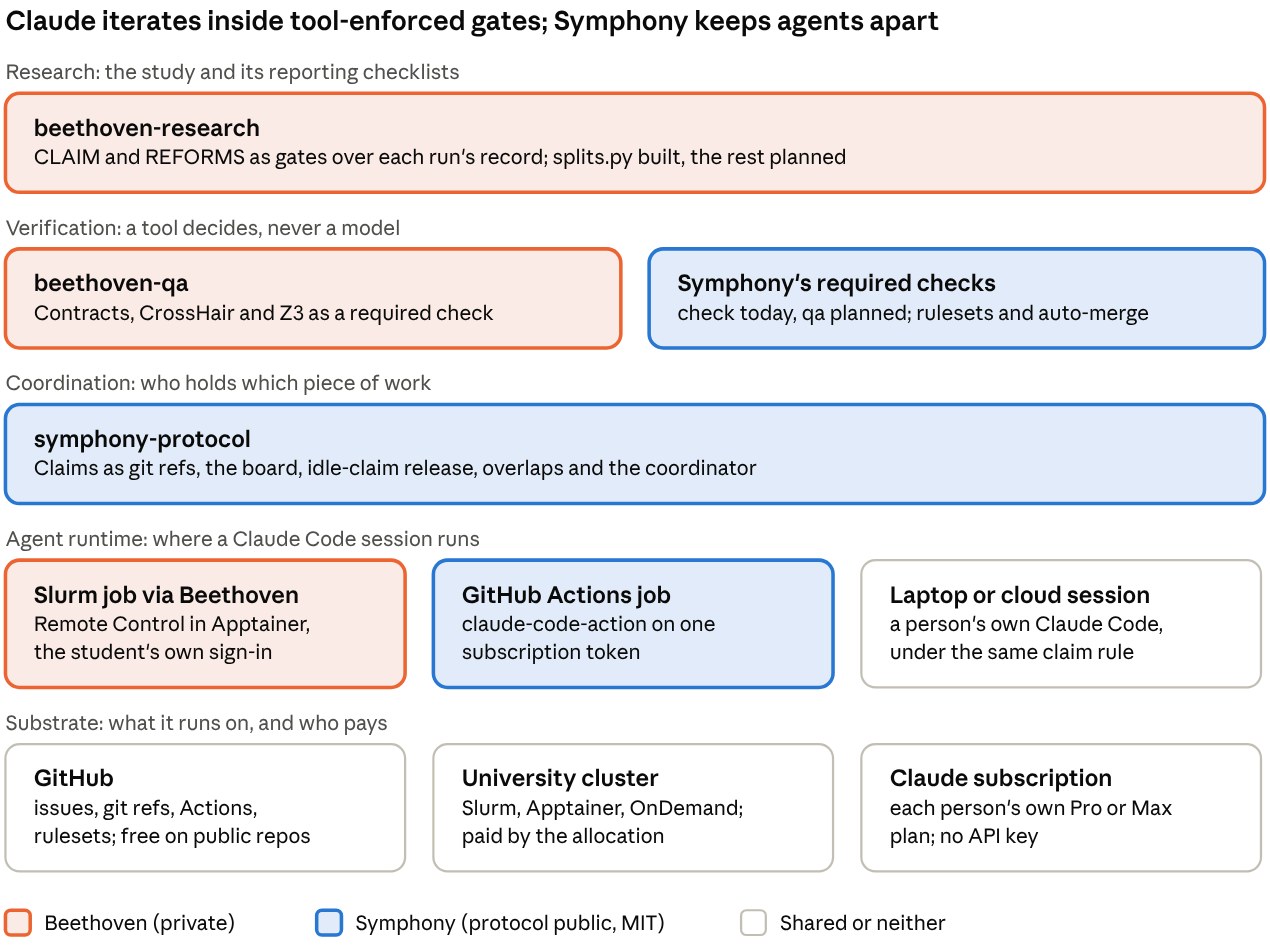
In the owner's words, beethoven-base and beethoven-sites hold the protocols for grid jobs on Wayne State's cluster, beethoven-research the reproducible CLAIM and REFORMS workflows, and beethoven-qa the checks on code that agents write

autonomously, while beethoven-agents and symphony-protocol coordinate the agents; Claude iterates the scientific method inside these frameworks. All of Beethoven is private and proprietary. The repositories' roles and their private, proprietary status are Verified (research notes 04 and 05). That symphony-protocol coordinates Beethoven's agents is the owner's design and is in no Beethoven file yet (Inferred; see the next subsection).

Part	What it does	Built so far
Launcher: hpcworker and the Chrome extension	Runs the student's own claude remote-control in tmux inside an Apptainer image as an ordinary Slurm batch job, reached through Open OnDemand; outbound HTTPS only, the student's own Pro or Max sign-in, no API key and no setup-token	Working at one site, Wayne State; extension 20.0.0, pre-release
beethoven-base, beethoven-sites	The shared library and the data-only site modules, so a new university is a pair of JSON files rather than code	Built
beethoven-qa	Contracts on every public function (icontract), CrossHair with Z3 as a required check, six code-standards groups reported as SARIF, auto-merge with no human gate	Version 5.0.0 on main, untagged (the last tag, v3.0.0, is what both consumers pin); the symbolic step confirms 13 of 165 conditions over all paths and searches the rest within 20 seconds each
beethoven-agents	The hermetic Apptainer image, Claude Code managed settings, a tool guard, and bee, an account switcher that never touches a credential	Image, settings, guard and bee built; Metaflow on Slurm, the fenced lease, the registry and the bus not built

Part	What it does	Built so far
beethoven-research	REFORMS (32 items, 20 as code gates and 12 declared) and CLAIM, as gates over each run's record	One module, splits.py, the split-level disjointness check, at the level each corpus declares: patient, study, cadaver or CT source volume (REFORMS 6b in a draft's lettering, Reported); the rest not built

How they compose into one stack



The stack · five layers, Symphony and Beethoven parts marked

Read from the bottom: the substrate is rented or owned by someone else, sessions run where the work is, Symphony decides who holds which piece of work, tools decide whether the work is acceptable, and the study's checklists decide whether the result may be reported.

The stack is the owner's design, and its composition is Inferred: no Beethoven repository references Symphony today, and Symphony's own plan rules that no Beethoven repository runs Symphony's code or holds a Claude token. The parts that can be shared are Symphony's Claude-free core (the branch-name claim, the board, the protocol check, the push guard, required checks and auto-merge), because Beethoven forbids a stored Claude token in CI and Symphony's Actions workers need one. Verified: research note 05.

3. The taxonomy

No published taxonomy answers the question this spec asks. The academic taxonomies of agents read for it (2024 to 2026) classify by architecture, control structure, interaction type, autonomy, environment, memory and domain; none has an axis for where the compute runs and who pays for it (Inferred from the dozen read, not a systematic review; research note 13). Industry categories moved in 2025 and 2026 from assistants to agents: Gartner's 2025 report was a Magic Quadrant for AI Code Assistants (Reported, from a news digest), and a single secondary page, possibly machine-written, reports a 2026 Magic Quadrant for Enterprise AI Coding Agents (Reported). Thoughtworks' Technology Radar Vol 34 (April 2026) named "agent-native project memory and task tracking" a new category, with Beads as its example, put a small team of coding agents in Assess and coding agent swarms in Caution (Reported, from a dataset of the radar). The CNCF landscape has an AI Agent category with eleven subcategories, none for coding agents or for coordinating them (Verified, landscape.yml).

The taxonomy below keeps the axes that recur in that literature and adds the one it lacks. An axis earns its place only if it separates the neighbors in section 5.

#	Axis	Categories	Built on
1	Stack layer	L0 model; L1 single-agent harness (Claude Code, Codex CLI); L2 agent framework or SDK; L3 coordination layer; L4 execution platform (sandbox, CI, HPC scheduler, hosted service); L5 domain workflow	The CNCF landscape; Sapkota et al. 2025; Anthropic's split of harness and sandbox
2	Autonomy, by the human's role	Operator, collaborator, consultant, approver, observer	Feng, McDonald and Zhang 2025; the SE 3.0 levels of Hassan et al. 2025
3	Control topology	Single agent; orchestrator and workers; hierarchy of planners; decentralized peers; shared medium (blackboard, ledger)	Guo et al. 2024; Tran et al. 2025; Anthropic's patterns
4	Communication medium	Direct messages (mailboxes, A2A); context protocol (MCP); shared task list or blackboard; traces in the work itself (issues, branches, labels); tuple space	Yang et al. 2025; Gelernter 1985; Nii 1986; Heylighen 2016 on stigmergy
5	Concurrency control over shared work	Partitioning only (file ownership); lock; lease (a lock with expiry, renewal and a fence); optimistic (private copy, validate at merge, retry); hybrid	Kung and Robinson 1981; Gray and Cheriton 1989; Chubby, 2006
6	Dependencies managed	Shared resource; prerequisite; transfer; usability; simultaneity; task and subtask	Malone and Crowston 1994

#	Axis	Categories	Built on
7	Execution environment	Developer machine; vendor cloud; CI runner; HPC batch job; self-hosted server	Guo et al.; Suter et al. 2025; GitHub's runner docs
8	Trigger and lifetime	Interactive; event-driven; scheduled; batch; long-running loop or durable execution	Suter et al.; the two modes of claude-code-action
9	State and memory	Stateless; context window; session log; project memory files; shared work ledger; lineage record	Sapkota et al.; CNCF State and Memory; Beads
10	Cost model and payer	Inference: metered API, flat subscription, vendor credits; compute: own machine, vendor, free CI minutes, institution's allocation; licence: open, open core, proprietary	New here, from GitHub's billing pages and Anthropic's pricing
11	Verification and governance	None; self-check; model as judge; deterministic gates (tests, required checks, formal methods); human approval; external certification	MAST; Thoughtworks' feedback sensors; Hassan et al.'s Merge-Readiness Packs
12	Domain and maturity	Software engineering task; scientific workflow; research software by kind, use and readiness level; business process	Liu et al. 2024; Suter et al.; the GI and de-RSE guideline after Hasselbring et al. 2024

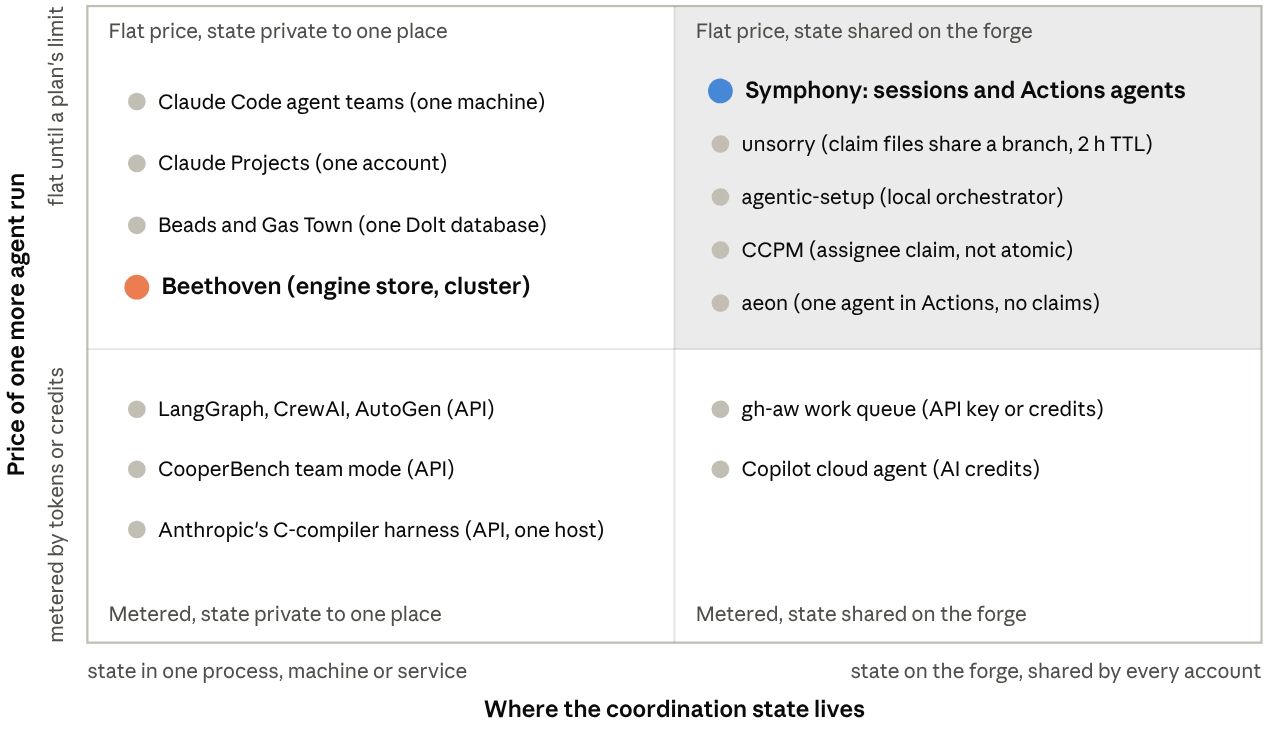
Three axes carry this study. Axis 10 is where the Actions story lives and the one no taxonomy has. Axis 5 is where Symphony's claim sits: a claim made by creating a branch, a lease-like release and optimistic merging. Axis 11 is where Beethoven's claim sits: a tool, never a model, decides. Axes 1, 3 and 7 locate the neighbors.

The classical frames fit closely. Malone and Crowston define coordination as "managing dependencies among activities" and name four families of dependency, one of which, producer and consumer, splits into prerequisite, transfer and usability (Reported, from mirrors of the paper; research note 13). Each of the six kinds has a counterpart in agent work: claiming an issue is a shared resource, a blocker is a prerequisite, a pull request is a transfer, a required check is a usability standard, the landing order of overlapping work is simultaneity, and an epic is task and subtask (Inferred; research note 13).

4. Placement

Two of the twelve axes separate Symphony from its neighbors at a glance: where the coordination state lives, and what one more agent run costs as each system is usually run. Positions inside a quadrant mean nothing; only the quadrant is a claim, and each is sourced in section 5.

Only Symphony puts agents in Actions under a claim at a flat price



Coordinators by where their state lives and what one more run costs · zones only, not measured positions

The tinted corner, a flat price with the state on the forge, holds five systems. unsorry, a Lean proof swarm, claims by pushing claim files to one shared branch, expires them with a two-hour timer in Actions and renders its board in Actions, but its agents run on

volunteers' machines. agentic-setup claims by creating a branch, but its orchestrator is a local session. CCPM claims by setting an assignee and a label, which two agents can both do. aeon runs one agent's scheduled skills in Actions with no claims between workers. Symphony is the only one there that puts agents running in Actions and people's own sessions under one claim rule (Inferred from research/taxonomy/verify/03, whose rows show none of the other four with such a rule; section 8). Below the line, the gh-aw work queue is Symphony's closest neighbor: it too makes a claim atomic with a compare-and-swap on a git ref, but a trusted scheduler grants claims only to Actions runs, a claim ends when its run ends rather than after idle time, and it sits there because its Claude engine takes an API key, workload identity or Copilot credits and refuses the subscription token (Verified, research/taxonomy/verify/01).

Placement on every axis:

#	Axis	Symphony	Beethoven	Nearest neighbors in the same cell
1	Stack layer	L3 coordination, on L4 GitHub	L4 HPC launcher and L5 research workflows	L3: openai/symphony, unsorry, CCPM, Beads, gh-aw's queue. L4 on HPC: Open OnDemand apps, VS Code tunnels in Slurm jobs, Academy
2	Autonomy	Sessions: collaborator to approver. Workers that auto-merge: observer, bounded by the required check	Student sessions: operator to collaborator, supervised by default. The maintainer clicks Publish	Copilot cloud agent and Devin at approver to observer
3	Control topology	Shared medium (the board) with self-claiming peers, and a planner for epics	A hierarchy under one lease holder (planned)	Cursor's planners, workers and judge; Claude Code agent teams; the C-compiler harness, peers with no orchestrator

#	Axis	Symphony	Beethoven	Nearest neighbors in the same cell
4	Communication medium	Traces in GitHub objects (issues, labels, branches, claim files, a rendered board); no agent-to-agent chat	An engine store and a bus (planned); Slurm	Beads′ git-backed ledger; gh-aw′s work-queue.jsonl on a branch; agent teams′ mailboxes
5	Concurrency control	Hybrid: a claim made by creating a branch, a lease-like release after 24 idle hours, a local fence on forced pushes, optimistic merge through the required check	A lease with a fencing token (planned)	Lock files in the C-compiler harness; unsorry′s claim files with a timer; agentic-setup′s create-only branch push; DND-IT/action-lock; Chubby; Claim Plane′s admission control
6	Dependencies managed	All six	Shared resource, task and subtask, simultaneity	Claude Code agent teams (task dependencies)
7	Execution environment	GitHub Actions runners, and each worker′s own session	A Slurm batch job in Apptainer	Copilot cloud agent and gh-aw on Actions; Anthropic′s long-running guide on Slurm with tmux
8	Trigger and lifetime	Event-driven, with a six-hourly schedule as a fallback	Batch; the worker queues its successor before its wall time	gh-aw; claude-code-action
9	State and memory	git and GitHub as the durable ledger; docs/memory as shared memory	Run records, an engine store, status files	Beads; Temporal

#	Axis	Symphony	Beethoven	Nearest neighbors in the same cell
10	Cost model and payer	Compute: free standard minutes on a public repository. Inference: one Claude plan for the Actions workers, each person's own for sessions. Protocol MIT, App paid	Compute: the university's allocation. Inference: the student's own plan. Proprietary	gh-aw: minutes plus API key or Copilot credits. Copilot agents: minutes plus AI credits. openai/codex-action: API key
11	Verification and governance	A deterministic required check; qa planned; the owner or auto-merge decides	Contracts and CrossHair with Z3 as a required check; REFORMS and CLAIM gates planned; a person clicks Publish	GitHub's security validation of agent code; Claude Code Review (advisory, never blocks); data-to-paper's number tracing
12	Domain and maturity	End-to-end software development; pre-release	Research software for a medical imaging study; mostly unbuilt beyond qa	The AI Scientist, Agent Laboratory, Kosmos

All placements are Inferred from research notes 01, 04 to 07 and 13.

5. Neighbors compared

5.1 Coordinators of coding agents

System	Where it runs	The claim	An idle claim	Workers	Inference paid by
Symphony	Actions, and each worker's own session	Creating branch claude/N; git lets one of two racing creations win	Released after 24 hours without a commit; the work is kept under a renamed branch	People's sessions, routines, agents in Actions	One Claude plan for the Actions workers; each person's own plan for sessions
gh-aw work queue	Actions	A record on a ledger branch, published by compare-and-swap; a trusted scheduler grants it	Never expires; ends with its run, then up to three attempts	Actions runs only	API key, workload identity or Copilot credits; the subscription token is refused
unsorry	Board, dispatcher and reaper in Actions; agents on volunteers' machines	Claim files pushed to one shared branch, first push wins, checked again after the push	Deleted two hours after the claim by a reaper every 15 minutes; the work is not kept	Volunteers' claude -p, Codex and Gemini runs	Each volunteer
agentic-setup	A local Claude Code orchestrator	Creating a branch, made create-only with a lease	Not reclaimed; resumed by the next round	Claude Code and Codex loops	The user
openai/symphony	A long-running daemon on a host	Held in the daemon's memory	A stall timeout kills the worker, then retries	Codex	The user
CCPM	A local Claude Code session	Assignee and label, which two agents can both set	None	Streams inside one session	The user's plan

System	Where it runs	The claim	An idle claim	Workers	Inference paid by
Beads and Gas Town	Local, with data synced to a git ref	Atomic inside one Dolt database, merged across machines after the fact	Released by hand (Beads); stuck agents detected (Gas Town)	Local agents	The user
paperclip	A self-hosted server	Atomic checkout on the server	Heartbeats and execution locks	Agents in an org chart	The user
Copilot cloud agent, Agent HQ	Actions runners, with GitHub's service as control plane	A person assigns; one agent, one branch, one pull request	A session stops at 59 minutes	Copilot, Claude and Codex agents	AI credits plus minutes; automations are not offered on public repositories
Claude Code agent teams, Projects	One machine; Anthropic's cloud for one account	A file lock on one machine; none across accounts	None	Teammates in one session; threads of one account	The user's plan
Anthropic's C-compiler harness	Containers on one host with a bare git upstream	Lock files that git refuses a second agent	None	16 Claude agents	API, just under $20,000

Sources: research notes 06 and 07, and research/taxonomy/verify/01 and 03 (Verified from each project's repository or docs, 2026-10-15).

5.2 Research automation

System	Runs where	How it checks its work	Reporting checklist as a gate	Paid by
Beethoven	A Slurm job on the university cluster, through Open OnDemand	Contracts with CrossHair and Z3 as a required check	Planned: REFORMS (20 items in code) and CLAIM; one check built	The student's plan and the allocation

System	Runs where	How it checks its work	Reporting checklist as a gate	Paid by
The AI Scientist v2	The user's GPU machine, in Docker	One model reviews the finished paper's text and a vision model its figures (Verified, the v2 launcher; v1 used an ensemble of five)	None	API, about $15 to $20 a run
Agent Laboratory	The user's machine	A person in the loop	None	API
Kosmos (Edison)	The vendor's cloud	Experts graded statements after the fact: 79.4% accurate	None	The user, about $200 a run at launch, with a free academic tier (Reported)
data-to-paper	The user's machine	Every number traced to the line of code that made it	None	API
Biomni	A local environment	None beyond its tools; runs model-written code with full privileges	None	Anthropic API key
Anthropic's long-running guide	A Slurm GPU node, in tmux	Tests before each commit, a changelog as lab notes	None	Plan or API, and the allocation

System	Runs where	How it checks its work	Reporting checklist as a gate	Paid by
Academy (Argonne); hpc-mcp and fractale (LLNL)	Research infrastructure, clusters	None of their own	None	The allocation
Snakemake, Nextflow, Metaflow	Clusters or clouds	Workflow structure only; no agent	None	The allocation

Sources: research note 11 (Verified for repositories and licences; Reported for figures from papers).

5.3 Verification of agent code and of protocols

System	What it checks	Can it block a merge?
symphony-protocol's harness	The shipped library, over every interleaving of scripted workers, against an oracle written from the spec's text, with 93 mutants	Its fast lane, a search to depth 5, is part of the required check; the deep searches run weekly or by hand, not as required checks
beethoven-qa	Contracts on every public function, by bounded symbolic search with CrossHair and Z3	Yes, as the required check, with no human gate
gh-aw's work-queue models	TLA+ models with 68 deliberately broken configurations; not the shipped Go and JavaScript	No
GitHub's security validation of agent code	CodeQL, secret scanning and the advisory database, on Copilot's and third-party agents' code, with no Advanced Security licence	It runs on the agent's own work

System	What it checks	Can it block a merge?
Copilot code review	A model's review	Only when Copilot approvals are switched on
Claude Code Review	Several model reviewers	Never: it neither approves nor blocks
SpecPylot	A model writes icontract contracts; CrossHair validates them	No: a generator, and it needs an API key
Specula; untyped	Agents write and model-check TLA+ of system code; a TLA+ model of one agent harness's tool calls	No

Sources: research note 12 and research/taxonomy/verify/03.

6. Integration

6.1 What the stack depends on

- **Symphony:** GitHub (issues, labels, refs, Actions, rulesets, environments, a GitHub App for the first commit and the sync), anthropics/claude-code-action@v1, Claude Code's hooks, and Python 3.12's standard library. Nothing else: the writing rules forbid new dependencies.
- **Beethoven:** Slurm, Apptainer and Open OnDemand at the university, Claude Code's Remote Control, Chrome for the extension, and icontract, CrossHair and Z3 for the gate; Metaflow with metaflow-slurm is chosen but not built, and metaflow-slurm is a 0.0.4 release from 2024 that drives sbatch over SSH with a key file, which conflicts with Beethoven's rule never to touch a credential unless its planned plain sbatch adapter replaces it (Verified, research note 11).

6.2 The interfaces, which are the integration surface

Symphony's interfaces are files and names in the repository, so anything that can push a branch, edit a label or read a file can take part.

Interface	Shape	Written by	Read by
Claim	Branch claude/N whose first commit adds docs/agents/claims/N.md (Worker, Plan, Files to touch)	The worker	The board, the clerk, the push guard
Labels	todo, claimed, in-review, blocker, epic, blocked, worker:name	Workers, the coordinator, people	The board, the clerk
Board	STATE.md on the branch board: queue, work given to workers, claims, overlaps	board.yml only	Every worker, every person
Worker registry	.github/symphony/workers/name.json: where it runs, its area, how its pull requests land, model and turns	People, the Add a worker form	workers.yml, the clerk
Coordinator comments	Marked first lines, for example the claim-released marker	The board′s run	The board (a release counts only from a writer)
Gate	The required check named check	CI	The ruleset, auto-merge
Cost record	A Symphony-Usage trailer on commits and a usage artifact per Actions run	Sessions and workflows	metrics.yml

Beethoven′s interface to GitHub is narrower: commit statuses named grid/... posted from the cluster, because GitHub′s runners cannot reach the grid, plus the gate′s SARIF reports (Verified, research note 11).

6.3 What it composes with

- **GitHub′s own gates** fold into Symphony′s planned qa check: CodeQL, dependency review and secret scanning, free on a public repository (Verified for CodeQL and secret scanning; GitHub′s own pages disagree on dependency review, as section 9.2 says; the qa check is not built).
- **beethoven-qa** is planned as the Python design-by-contract lane of that qa check (a recorded decision; not built).

- **Other coding agents** could claim under the same rule if they push claude/N with a claim file: the protocol is agent-neutral in shape, but Copilot's cloud agent and gh-aw name their own branches, so each would need an adapter (Inferred; Open).
- **gh-aw** could run as a kind of Symphony worker, its safe outputs opening the pull request, or Symphony's board could read gh-aw's queue; neither exists (Inferred; depends on section 5's comparison).
- **GitHub's merge queue** can land work after Symphony orders overlapping pieces; the two are complementary (Inferred).
- **Workflow managers** for Beethoven's outer loop: Snakemake (with a Slurm executor plugin), Nextflow, Parsl or CWL are alternatives to the chosen Metaflow; DVC for data versions; MLflow or Weights and Biases for run records (Verified as products, research note 11).
- **The Symphony plugin** in symphony-marketplace brings the skills, /board and the hooks to a repository not made from the template.

6.4 What it could replace

- A coordination server of the project's own (a paperclip server, a Beads Dolt server, an openai/symphony daemon) for a team whose work already lives on GitHub (Inferred).
- Convention-only claiming, such as CCPM's assignee and label, which two agents can set at once (Inferred, research note 07).
- For Beethoven, the checklist tables authors fill after the fact, if the gates are built and shown to catch what reviewers miss (Open, question R8).

6.5 Where Symphony and Beethoven meet today

Nowhere in code. Symphony's plan says no Beethoven repository runs Symphony's code or holds a Claude token; a Markdown-only pilot of the claim protocol in beethoven-qa is an open owner question, recommended yes after Symphony's Phase 3 (Verified, research note 05). The grant packet puts its agent under the laboratory's claim protocol of one issue, one branch, one pull request, and names neither Symphony nor symphony-protocol (Verified, research note 05). That protocol matches Symphony's rule without its Actions workers (Inferred: spine-program, the repository the packet builds on, adopted Symphony's protocol alone on 2026-10-12, according to the template's checkpoints).

7. The need

Three problems, each with its evidence. The strength of each source is stated, because the evidence is uneven.

7.1 Agents that share a codebase collide, and chat does not fix it

- **The curse of coordination.** CooperBench (Stanford and SAP Labs, preprint, 2026) gave 652 feature pairs from 12 repositories either to one agent or to two agents in isolated workspaces that could only message each other. One agent passed far more often: GPT-5 315 against 183 of 652 (48.3% against 28.1%), Claude Sonnet 4.5 307 against 168 (47.1% against 25.8%). The paper states the gap three ways (30% in the abstract, up to 50% in section 4, 41% in appendix C), so quote the counts. Verified, the paper's PDF in the cooperbench/website repository.
- **What fails is duplicated or incompatible work, not skill.** Of the failed two-agent runs, 33.2% were work overlap and 29.7% divergent architecture; in 50 hand-read traces the root causes were expectation (42%), commitment (32%) and communication (26%). Messaging took up to 20% of the agents' steps and did not raise success. A plan in the first turn nearly halved the conflict rate (29.4% against 51.5%). Verified, same paper.
- **Git alone is not enough; a shared substrate with a task list is.** Giving each pair a git server moved success by only 1 to 3 points (Verified, the February 2026 post in the cooperbench/website repository). In CooperBench's later ablation (50 pairs, one seed, one model) messaging alone reached 26%, one agent 48%, a shared git remote 56%, and a lead with a shared task list and scratchpad 62%; removing the task list or the scratchpad dropped the team below one agent. Weak evidence (n = 50, one seed), but it points the same way. Verified, the CooperBench repository at 63b9d44.
- **The field shows the same.** Merge-simulating 107,026 agent pull requests found textual conflicts in 27.67% (AgenticFlict, Verified), and 40.2% of popular repositories in the AIDev dataset had agent pull requests open at the same time (Reported).
- **Practitioners converge on claims, ownership and gates.** Anthropic's 16-agent C compiler claimed tasks by writing lock files that git refused a second agent (Verified); Claude Code agent teams claim tasks under a file lock and advise each teammate to own different files (Verified); Cursor reports that naive locks collapsed 20 agents to the throughput of two or three, because locks were held too long or never released (Reported).
- **The owner's own replication** (coordbench) has run 8 pairs: Symphony's arm passed 6 of 8 like every two-agent arm, one agent 5 of 8, and no difference is significant (every exact McNemar p is 1.000; Verified, research note 03). It is a pilot, and its token and

dollar figures are inflated by a prompt of about 78,000 to 82,000 tokens that every agent there inherits (Verified, research note 03). It stays out of the grant until you say so.

What each Symphony primitive answers, by design (Inferred, not measured):

Failure in the evidence	Symphony's answer
Two agents build the same thing (33.2%)	One claim per issue; the second creation of claude/N is refused
An agent assumes its partner's work does not exist (42%)	Claim branches and draft pull requests on one remote, and a board with an Overlaps section
Promises not kept, contracts unverifiable (32%)	The pull request's diff and the required check are the commitment
No plan before coding	The claim file, with the plan and the files to touch, is the first commit
Agents misuse git (force pushes, overwrites)	The push guard refuses forced pushes, and leases onto another worker's claim
Locks never released (Cursor)	The board releases a claim idle for 24 hours and keeps its work (idle by the commit's own date; risk 13)

The gap: no peer-reviewed study yet compares a claim- and pull-request-gated protocol on a shared GitHub remote with one agent at a matched budget, and a 2026 noise-floor study puts single-seed gains under about 15 points inside run-to-run noise (Reported). Section 11 makes this the first research question.

7.2 Agent-driven research fails where the paper cannot show it

- Luo, Kasirzadeh and Shah (NeurIPS 2025) found data leakage, inappropriate benchmarks, metric misuse and post-hoc selection in two AI-scientist systems, and found them far easier to detect from trace logs and code than from the paper (Reported).
- Beel and colleagues found 5 of 12 proposed experiments (42%) failed on coding errors, with some hallucinated numbers (Reported; n = 12). Kosmos reports 79.4% of its statements judged accurate, so about one in five is not (Reported).

- Reporting checklists are filled by authors after the fact: a 2025 umbrella review put median CLAIM compliance at 68% across 421 studies, and in 2024 only 5 of 98 imaging journals encouraged AI-specific reporting guidelines (Reported). No AI-scientist system found enforces REFORMS, CLAIM, TRIPOD+AI or a similar checklist as a gate (Inferred; an absence claim from a search that ran out of budget).
- Code written by agents is often insecure: on SusVibes (ICML 2026), Claude Opus 4.8 in Claude Code passes 83.9% of tasks functionally but only 32.8% both functionally and securely (Verified from the leaderboard's submission files; the entries are self-submitted).

This is the gap Beethoven's design names: gates that read the agent's run record, not its paper, and fail the run when a checklist item is unmet. It is almost entirely unbuilt (section 2).

7.3 Cost and setup keep coordination out of reach

- Coordination multiplies tokens: Anthropic measured multi-agent research systems at about 15 times the tokens of chat, and Claude Code's agent teams at about 7 times a standard session in plan mode (Verified, research system, costs). The 16-agent C compiler cost just under $20,000 in API fees over two weeks (Verified).
- The neighbors that run agents in CI meter every run: gh-aw's Claude engine takes an API key or Copilot credits, openai/codex-action needs an API key, and Copilot's agents spend AI credits at $0.01 each as well as Actions minutes (Verified, research note 10), though GitHub's docs do not say whether the cloud agent's minutes are free on a public repository (research/taxonomy/verify/01).
- Free CI for open source has shrunk everywhere but GitHub: Travis CI ended unlimited open-source builds in 2020, GitLab.com put public projects on quotas from 2021, Azure DevOps stopped new public projects in 2026 and will make the existing ones private in 2027, pointing them to GitHub, and Cirrus CI stopped running jobs on 2026-06-01 (Verified from their docs for GitLab's quotas today and for Azure; Reported for GitLab's 2021 start and the rest). Section 9 takes this up.

8. What is unique

Every single mechanism Symphony uses has prior art. What may be unique is the combination, and three of its parts. Each claim below names the observation that would refute it. Symphony's three claims, U1 to U3, were put to an adversarial search of 25 candidates on 2026-10-15 (research/taxonomy/verify/03); none had four of the five

mechanisms below, but one, unsorry, has (c) and looser forms of (a), (b) and (d), so the claims are worded narrowly. Beethoven's three, U4 to U6, were not part of that search; they rest on narrower searches in research notes 04, 11 and 12.

The five mechanisms: (a) a claim made atomic by a git ref on the shared forge; (b) automatic release of an idle claim by a CI job, keeping the work; (c) a board and coordinator that run as CI jobs with state only in forge objects; (d) people's own sessions and agents in CI under one claim rule, on a flat-rate subscription; (e) validation of the shipped protocol code by exhaustive interleavings against an oracle written from the spec, with mutants.

Mechanism	Prior art
(a)	terraform-backend-git and actions-mutex (2020), DND-IT/action-lock (2026), Anthropic's C-compiler harness (2026-02), agentic-setup's create-only branch push, gh-aw's ledger and unsorry's claims branch
(b)	Leases (Gray and Cheriton, 1989); zulipbot, a hosted bot, unassigns an issue whose assignee does not answer within 3 days of a prompt sent after a week without an update; unsorry's two-hour reaper in Actions (2026-06); DND-IT/action-lock (2026-02) frees a CI lock held over 600 seconds by its commit's date, the clock Symphony reads, with no work to keep; agentsync flags claims idle for 24 hours as stale without freeing them
(c)	unsorry (2026-06) and gh-aw's work queue (2026-10)
(d)	aeon runs one agent in Actions on the subscription token, with no claims; claude-code-action itself
(e)	gh-aw checks TLA+ models of its queue, not its code; TLA+, P and Stateright are the established techniques

The claims

#	Claim	Grade	Refuted by
U1	Symphony is the only system found that puts people's own Claude Code sessions and agents running in GitHub Actions on a subscription token under one claim rule, with the claim, board, release and coordinator kept in GitHub objects and run by Actions	Inferred, from the search above	A system with four of (a) to (e), or with (a), (c) and (d) together
U2	It is the only system found that releases a claim by idleness since its branch's last commit and keeps the work under a renamed branch	Inferred	A system that releases by last activity and keeps the work. Caveat: the idle clock is the pusher's (spec risk 13)
U3	It is the only coordination protocol for agents found whose shipped code, not a model of it, is checked over every interleaving against an oracle written from the spec's text, with mutants that prove the checks bite	Inferred; the search was narrow (no paper databases)	A coordination layer for agents whose shipped code is model-checked
U4	Beethoven is the only system found that plans agents as Slurm jobs on the student's own subscription, reporting checklists as gates over each run's record, and contracts checked symbolically, together	Inferred; an absence claim from a search that ran out of budget; mostly unbuilt	Any system that gates REFORMS, CLAIM or a similar checklist in code

#	Claim	Grade	Refuted by
U5	Beethoven's launcher is the only tool found that starts Claude Code's Remote Control from an sbatch job through Open OnDemand, with nothing installed by an administrator	Inferred	A center-run Remote Control app for Open OnDemand
U6	beethoven-qa is the only project found in a narrow search of public repositories that makes contracts plus symbolic execution the required merge gate for code agents write	Inferred; a narrow search; the gate is a bounded search for most conditions, not a proof	Another required gate of this kind

What is not unique, and should not be claimed

- An atomic claim on the forge: gh-aw, unsorry and agentic-setup have one.
- Coordination that runs in Actions with no server: unsorry and gh-aw do it too.
- Free coordination on a public repository: any of them gets free minutes there.
- The subscription token in Actions: aeon and anyone using claude-code-action have it.
- One issue, one branch, one pull request: Copilot's cloud agent works the same way.
- The stack of Symphony and Beethoven: it is a design, not yet code (section 2).

9. GitHub Actions nativeness and free public minutes

9.1 What "Actions-native" means here

Symphony's coordinator is not a service that talks to GitHub; it is a set of GitHub Actions jobs that run when GitHub says something happened. The board, the idle-claim release, the overlap notices, the dealing of work to workers, the protocol check and the cleanup are deterministic code on the workflow's own GITHUB_TOKEN, with no Claude call and no server. GitHub's own events trigger them, GitHub's concurrency queue serializes them,

GitHub's refs make the claim atomic, and GitHub's rulesets and required checks enforce the result. Claude runs only where judgment is needed, as a job on the same runners. Verified, research note 01.

9.2 What is free

- GitHub's docs: "Use of the standard GitHub-hosted runners is free and unlimited on public repositories" (supported-github-runners.md, Verified 2026-10-15). Self-hosted runners are free on every plan.
- A public repository gets twice the machine a private one does: Linux and Windows standard runners with 4 vCPUs, 16 GB and a 14 GB SSD, against 2 vCPUs and 8 GB in a private repository. Verified.
- The governance a coordination protocol needs is free there too: rulesets, required checks and code owners on GitHub Free, environments with required reviewers and wait timers (which on Free, Pro and Team exist only for public repositories), CodeQL code scanning, secret scanning with push protection, and Dependabot. Symphony's gate and its owner-approved release environment rely on this. Verified; GitHub's own pages disagree on whether dependency review is free on a public repository, and most say it is.
- GitHub announced a $0.002-a-minute charge on self-hosted runners on 2025-12-16 and withdrew it from the docs the next day; secondary sources say it is postponed, not cancelled (Verified from the github/docs history; Reported for the postponement).

9.3 What "unlimited" does not mean

Limit	Value	What it means for Symphony
Concurrent jobs on standard runners, per account	Free 20, Pro 40, Team 60, Enterprise 500	Caps how many Actions workers one account runs at once, across all its repositories
One job	6 hours; a run 35 days	A worker's run must finish or hand off within 6 hours
GITHUB_TOKEN	1,000 REST requests an hour per repository	Likely binds the board and coordinator before minutes do; not yet measured (R3)

Limit	Value	What it means for Symphony
A queue: max concurrency group	100 pending runs; the rest are cancelled	The board's queue can overflow under a burst of events
Content-creating requests	About 80 a minute and 500 an hour	Limits comments and labels in a burst
Schedules	Disabled in a public repository after 60 days without activity; delayed at the top of the hour	Symphony's schedules are fallbacks only; events drive it
Larger runners and GPUs	Always billed, even on public repositories	No free GPU; Beethoven's science cannot run on Actions
Cache	10 GB per repository; entries unused for 7 days are evicted	Records kept in the cache must tolerate eviction

The limits are Verified from the github/docs repository at 12691d2 (2026-10-15), research note 09. What each means for Symphony is Inferred, including that the concurrency caps apply to public repositories (GitHub's table is by plan) and which limit binds first, which question R3 measures. GitHub has narrowed free public use only to fight abuse (approval for first-time contributors in 2021, holds on suspicious runs in 2026, a default block on pull_request_target from 2026-11-02), never by metering standard minutes (Inferred).

9.4 What still costs money

- **Inference is never free.** Symphony's Actions workers run on one person's Claude plan through CLAUDE_CODE_OAUTH_TOKEN (Pro $20 a month, or $17 billed annually; Max $100 or $200), and they share that plan's five-hour window and weekly limit with the person's own use. The Max and Team monthly API credits do not cover runs of the Claude Code GitHub Action. Verified, research note 10.
- **A private repository** gets 2,000 included minutes a month on Free, 3,000 on Pro and Team, 50,000 on Enterprise Cloud, then $0.006 a minute on Linux (since 2026-01-01), each job rounded up to a whole minute, and usage stops at the quota without a payment method. Rulesets that enforce the gate need Pro or Team there. Verified.

- **An illustration** (Inferred; the workload is an assumption): ten one-hour Claude runs a day is 18,000 runner minutes a month. On a public repository that costs nothing in minutes. On a private repository on Team it is 15,000 minutes over the quota, about $90 a month, before the board's own runs, each of which bills at least a minute.
- **AI review layers** (Copilot code review, GitHub's AI Scan, Claude Code Review) bill credits even on public repositories; Symphony's rules keep them out of the gate. Verified, research note 12.

9.5 The terms that govern it

- GitHub's Actions terms (effective 2026-04-27) forbid offering Actions "or any elements of the Actions product or service, for commercial purposes", disproportionate burden, and, on GitHub-hosted runners, "any other activity unrelated to the production, testing, deployment, or publication of the software project associated with the repository" (Verified, github/site-policy). Agents writing and testing the repository's own code sit inside that line (Inferred). A benchmark that runs other projects' tasks, and a paid App whose runtime rides each customer's free minutes, need a reading (Open, question R5).
- Anthropic documents the subscription token for claude-code-action and says runs then "use your Claude subscription instead of API billing", but advises an API key for a secret shared across repositories, since the token is tied to one person's plan; its legal page says plan limits "assume ordinary, individual usage" (Verified, GitHub Actions, legal and compliance). Whether several Actions workers on one plan are ordinary, individual usage is not stated (Open, question R6).
- Claude Code's docs say bare mode will become the default for claude -p and that bare mode does not read the OAuth token; whether claude-code-action will keep the subscription path working is not stated (Open, question R6).

9.6 Why it matters

1. **A coordination step costs nothing.** Claiming, releasing, ordering overlaps and rendering the board are runner minutes on GITHUB_TOKEN, free on a public repository, with no model call. Neighbors that coordinate with a model, or through a hosted service, pay for every step.
2. **A Claude run costs plan usage, not dollars.** Until the plan's window is reached, one more worker run has no metered price. Of the neighbors examined that run agents in Actions, gh-aw, openai/codex-action and Copilot's agents meter inference by the token

or the credit; aeon runs one agent on the subscription token but coordinates no claims (sections 5 and 8).

3. **There is nothing to host.** No server, database or daemon of the project's own: the forge is the lock, the CI is the scheduler, and the repository is the state. That removes the setup and the bill that keep a coordinator out of a small project, and it survives the churn of agent frameworks (section 5).
4. **The offer is GitHub's alone, so far.** No other major hosted CI found still offers new public projects unlimited free minutes (Azure Pipelines keeps them only for existing public projects, until those turn private in 2027): section 7.3 lists who withdrew, and CircleCI caps open source at 400,000 credits a month (Reported, from its pricing page by search). That is also a dependency: one vendor's pricing decides the economics (section 10).
5. **It does not reach Beethoven as built.** Beethoven's repositories are private and its science needs the cluster and GPUs, which Actions never gives free. Actions fits Beethoven as the control and verification plane, with the cluster as the compute plane, as its CI already does: the gate runs on GitHub's runners and a grid lane posts commit statuses back. A public Beethoven gate library would run free (Inferred).

10. Gaps, risks and open questions

#	Risk	Why it matters	What would show it, or reduce it
1	One vendor sets the economics	Free public minutes, free governance and the ref semantics are GitHub's to change; the Actions terms limit use to the repository's own software project	GitHub's pricing and terms pages; a self-hosted runner as the fallback

#	Risk	Why it matters	What would show it, or reduce it
2	The subscription token in CI	Anthropic documents it but advises an API key for a shared secret, its terms assume ordinary, individual usage, its rules for subscriptions moved several times in 2026, and bare mode will stop reading the token	Questions R6; keeping an API-key path in the workflows
3	One plan's window for every Actions worker	All workers share one person's five-hour and weekly limits; this project's own account reached its weekly limit on 2026-10-15 with several threads running	The planned cap of 10 Claude runs a day; one plan per worker
4	Symphony's own tests run private for now	Its plan runs the owner's first fresh test on private repositories only until the Phase 3 rails exist (the run cap, the qa check, checks pinned to Actions), though it recommends public repositories for the code workers change where possible; so Symphony's own runs have not yet tested the free-minutes story	Phase 3 landing
5	GitHub's rate limits before minutes	GITHUB_TOKEN gets 1,000 requests an hour per repository and the library has no retry on 429 or secondary limits yet	The measurement pass (question R3)

#	Risk	Why it matters	What would show it, or reduce it
6	Maturity	The library is days old, 0.4.1 is the last release, later fixes are unreleased, and no full-scale benchmark of Symphony exists	Question R1; the next release
7	The name	openai/symphony (about 27,700 stars, Apache-2.0) also claims issues for coding agents	Question R10
8	Beethoven's evidence is mostly plans	Only splits.py of the research gates exists; the symbolic gate is a bounded search for most conditions, not a proof	Question R8
9	Beethoven's setting	Wayne State requires approval of AI tools; a personal plan has no BAA; Remote Control is unavailable under the HIPAA configuration and stores transcripts on Anthropic's servers	Written answers from the university and Anthropic, already preconditions of any paid edition
10	Prompt injection through issues	Claude workers in Actions read issue text and hold write tokens	Recorded decisions: only writers queue work, code owners guard the files that steer workers
11	The public story is untold	The site and READMEs never mention Actions, free minutes or cost; several internal documents lag GitHub's current behavior	The manual's talking points; a pass over the docs

#	Risk	Why it matters	What would show it, or reduce it
12	The server enforces a claim only at its creation	Git refuses a second creation of claude/N, but GitHub accepts a fast-forward or forced push onto an existing claim under the recommended setup, since rulesets must leave claude/** alone. Only the local push guard refuses forced pushes, and on 2026-10-15 it was found to read the session's own directory rather than the clone a git -C command names, so a lease onto another worker's claim passed when run from a directory whose clone had neither branch, though it was refused from either worker's own clone	research/taxonomy/verify/02; a fix to the guard; question R4
13	The idle clock is the pusher's	Idleness is read from the head commit's committer date, which the pusher writes: a commit dated in the future holds a claim indefinitely, against the coordinator and a person's release alike, and one dated in the past is released at once. This is finding F8, open in 0.4.1 and on main	The unmerged F8 fix reads GitHub's record of the push, falling back to the commit date when that record is missing; question R4

#	Risk	Why it matters	What would show it, or reduce it
14	A reopened issue can be orphaned	The labels claimed and in-review stay on a closed issue; if it is reopened, the board lists it nowhere and no worker is dealt it until a person takes them off	A board rule for reopened issues; question R4

Open questions that are not risks:

- Whether a public, Claude-free Beethoven gate library is wanted, which is the only way Beethoven gets free minutes.
- Which subset of the taxonomy's twelve axes a short public explanation should use (the manual proposes three).
- Whether the experiment, once complete, belongs in this spec; it stays out of the grant until you say so.

11. Research questions

Each question owns the Open claims named in it. A question is answered only when its acceptance criteria are met; until then the claim keeps its grade.

#	Question	Answered when
R1	Does Symphony's protocol beat one agent, and the other two-agent arms, at a matched budget?	The coordbench full run you chose on 2026-10-15 (35 new pairs under five arms, plus the runs to redo) reports pass rates with exact tests and confidence intervals, and tokens, wall time and list-price dollars per pair; any gap under the roughly 15-point noise floor is reported as not established

#	Question	Answered when
R2	Is the combination in section 8 unique?	A documented search (FSE, ICSE and ASE 2026, arXiv cs.SE and cs.MA, GitHub topics, the awesome lists of agent orchestrators) scores every candidate on the five mechanisms of section 8; claim U1 stands only if none has four of the five, or (a), (c) and (d) together
R3	What does one coordination step cost, and which GitHub limit binds first?	The canary measures API calls per board run, the medians from claim to first pull request and from open to merge, and the number of concurrent workers at which GITHUB_TOKEN's 1,000 requests an hour or the 100-run queue gives out; the library retries on 429 and secondary limits
R4	Does the claim hold on github.com as it does on a local git server?	The canary records a second creation of claude/N, a push that descends from another worker's claim, a forced push with and without a ruleset, the rename on release, and a push after a merge; the clock question of F8 is settled
R5	Are Actions workers, benchmarks and a paid App inside GitHub's Actions terms?	GitHub's written answer, or a lawyer's reading, on three cases: Claude workers writing the repository's own code; coordbench running other projects' tasks; SymphonyBoard's open runtime running in customers' Actions
R6	Can several Actions workers run on one person's subscription token, and will that keep working?	Anthropic's written answer on a token in an organization secret, several parallel workers on one plan as ordinary, individual usage, and claude-code-action under bare mode; and a tested fallback that uses an API key

#	Question	Answered when
R7	What are free public minutes worth to an adopter?	For three workloads (one maintainer, a small team, a research lab), the monthly minutes and their price on a private repository, and the same work's price on gh-aw with an API key and on Copilot's agents in credits, with every assumption stated and every price linked
R8	Do Beethoven's checklist gates catch what reviewers miss?	REFORMS and CLAIM gates built for the first study catch seeded faults of the kinds Luo et al. found (leakage, post-hoc selection, metric misuse) that a reading of the paper alone misses; and a search names any other system that gates these checklists in code, or finds none
R9	Does the taxonomy separate the field?	Every system in section 5 is placed on all twelve axes with sources, no two systems practitioners treat as different share all twelve cells, and a systematic search confirms or refutes that no published taxonomy has a compute-payer axis
R10	How much does the name cost?	The first page of results on the main search engines and registries for Symphony with agents, coding or Claude is captured, with a recommendation: keep it, qualify it, or rename

The manual's section on deep research turns these into one message to paste.

12. Sources

The research behind this spec is in this project's files, under research/taxonomy/notes/ (thirteen notes, one per topic, each with its sources, line numbers and grades) and research/taxonomy/verify/ (the adversarial checks). Every claim above names its note. The primary sources most claims rest on:

Symphony and Beethoven

- symphony-protocol, its validation design and the run of record
- The template's PROTOCOL.md and free-or-paid.md
- The Beethoven repositories (private): hpcworker, beethoven-extension, beethoven-qa, beethoven-agents, beethoven-research

GitHub

- Billing for Actions, runners, limits and Advanced Security, read from the github/docs repository at 12691d2 (2026-10-15), because docs.github.com is unreachable from the research session
- The Terms for Additional Products and Features, effective 2026-04-27
- github/gh-aw at dc91dbed

Anthropic

- Claude Code GitHub Actions, Legal and compliance, Costs, Agent teams, Remote Control, pricing
- Building a C compiler with a team of parallel Claudes, How we built our multi-agent research system, Building effective agents

Evidence on coordination and agent code

- CooperBench: arXiv 2601.13295, code, website and paper PDF
- MAST, AgenticFlict, AIDev, SusVibes
- DORA 2025

Neighbors

- openai/symphony, CCPM, Beads, Gas Town, paperclip, Orbi, anthropics/claude-code-action, openai/codex-action

Research reporting and HPC

- REFORMS, Science Advances 2024; CLAIM 2024, Radiology: Artificial Intelligence; TRIPOD+AI, BMJ 2024
- Open OnDemand, Apptainer, Slurm
- Luo, Kasirzadeh and Shah, arXiv 2509.08713; The AI Scientist; data-to-paper

The taxonomy's own sources are linked in section 3.